

Intermediate (measure the sequential version) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Read both depths — they're not "wrong, right," they're 2 real, valid ways to solve the same problem, with real tradeoffs between them.

Basic Version

Approach 1 — the direct way

```
# architecture_comparison_practice.py – Intermediate section
import time

def research(topic):
    start = time.time()
    prompt = "Research the top 3 causes of " + topic + ". List them briefly."
    response = model.invoke(prompt)
    seconds = time.time() - start
    tokens = response.usage_metadata["total_tokens"]
    return response.content, tokens, seconds

def write(research_text):
    start = time.time()
    prompt = "Write a 3-sentence summary of this:\n\n" + research_text
    response = model.invoke(prompt)
    seconds = time.time() - start
    tokens = response.usage_metadata["total_tokens"]
    return response.content, tokens, seconds

topic = "climate change"
research_text, research_tokens, research_seconds = research(topic)
summary, write_tokens, write_seconds = write(research_text)

total_tokens = research_tokens + write_tokens
total_seconds = research_seconds + write_seconds

print("research:", research_tokens, "tokens,", research_seconds, "seconds")
```

```
print("write:", write_tokens, "tokens,", write_seconds, "seconds")  
print("total:", total_tokens, "tokens,", total_seconds, "seconds")
```

Expected output (exact numbers vary by run):

```
research: 210 tokens, 1.84 seconds  
write: 96 tokens, 0.91 seconds  
total: 306 tokens, 2.75 seconds
```

This works and gives you real numbers. It's missing a clean result type and doesn't separate "measuring" concerns from "doing the work" concerns — both fine for a first pass.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — type hints and a dict report

```
# architecture_comparison_practice.py – Intermediate section
import time

def research(topic: str) -> dict:
    start = time.perf_counter()
    prompt = f"Research the top 3 causes of {topic}. List them briefly."
    response = model.invoke(prompt)
    elapsed = time.perf_counter() - start
    return {
        "text": response.content,
        "tokens": response.usage_metadata["total_tokens"],
        "seconds": elapsed,
    }

def write(research_text: str) -> dict:
    start = time.perf_counter()
    prompt = f"Write a 3-sentence summary of this:\n\n{research_text}"
    response = model.invoke(prompt)
    elapsed = time.perf_counter() - start
    return {
        "text": response.content,
        "tokens": response.usage_metadata["total_tokens"],
        "seconds": elapsed,
    }

def main():
    topic = "climate change"
    total_start = time.perf_counter()

    research_result = research(topic)
    write_result = write(research_result["text"])

    total_seconds = time.perf_counter() - total_start
    total_tokens = research_result["tokens"] + write_result["tokens"]

    r_tokens, r_seconds = research_result["tokens"], research_result["seconds"]
    w_tokens, w_seconds = write_result["tokens"], write_result["seconds"]
```

```

    print(f"research: {r_tokens} tokens, {r_seconds:.2f}s")
    print(f"write: {w_tokens} tokens, {w_seconds:.2f}s")
    print(f"total: {total_tokens} tokens, {total_seconds:.2f}s")

main()

```

Expected output (exact numbers vary by run):

```

research: 210 tokens, 1.84s
write: 96 tokens, 0.91s
total: 306 tokens, 2.75s

```

Difference from Basic: full type hints (`topic: str -> dict`), `time.perf_counter()` instead of `time.time()` (the correct tool specifically for measuring elapsed durations), and each stage returns a dict with named fields instead of a loose 3-item tuple — `result["tokens"]` reads clearly at the call site, where `result[1]` from Basic doesn't say what it is without checking the function. Still no shared, reusable structure across stages — that's what Approach 2 adds.

Approach 2 — a typed `StageResult`, and a dedicated `run_sequential()`

```

# architecture_comparison_practice.py – Intermediate section
import time
from dataclasses import dataclass

@dataclass
class StageResult:
    text: str
    tokens: int
    seconds: float

def research(topic: str) -> StageResult:
    start = time.perf_counter()
    prompt = f"Research the top 3 causes of {topic}. List them briefly."
    response = model.invoke(prompt)
    elapsed = time.perf_counter() - start
    call_tokens = response.usage_metadata["total_tokens"]
    return StageResult(response.content, call_tokens, elapsed)

def write(research_text: str) -> StageResult:
    start = time.perf_counter()
    prompt = f"Write a 3-sentence summary of this:\n\n{research_text}"

```

```

response = model.invoke(prompt)
elapsed = time.perf_counter() - start
call_tokens = response.usage_metadata["total_tokens"]
return StageResult(response.content, call_tokens, elapsed)

def run_sequential(topic: str) -> dict:
    total_start = time.perf_counter()
    research_result = research(topic)
    write_result = write(research_result.text)
    total_seconds = time.perf_counter() - total_start

    return {
        "research": research_result,
        "write": write_result,
        "total_tokens": research_result.tokens + write_result.tokens,
        "total_seconds": total_seconds,
    }

if __name__ == "__main__":
    report = run_sequential("climate change")
    r = report["research"]
    w = report["write"]
    print(f"research: {r.tokens} tokens, {r.seconds:.2f}s")
    print(f"write: {w.tokens} tokens, {w.seconds:.2f}s")
    tt, ts = report["total_tokens"], report["total_seconds"]
    print(f"total: {tt} tokens, {ts:.2f}s")

```

Expected output (exact numbers vary by run):

```

research: 210 tokens, 1.84s
write: 96 tokens, 0.91s
total: 306 tokens, 2.75s

```

A small `StageResult` type instead of returning three loose values per function — the caller can't accidentally read `result.seconds` where it meant `result.tokens`, the way a bare tuple invites. A separate `run_sequential()` function keeps "run the whole pipeline and report on it" separate from each stage's own logic, so either can be tested or reused on its own — `supervisor_compare` reuses `research()` and `write()` unchanged, and builds a matching `run_supervisor()` report with the exact same 4 keys, so the two runs compare directly.

Approach 3 — running multiple topics and reporting the ratio

Once you have one topic's numbers, the next real question — the one `sequential_measure`'s Hint 2 asks you to answer — is whether research or write is the more expensive stage, and by how much, across more than one run.

```
# architecture_comparison_practice.py – Intermediate section
def run_sequential_batch(topics: list[str]) -> list[dict]:
    reports = []
    for topic in topics:
        reports.append(run_sequential(topic))
    return reports

if __name__ == "__main__":
    topics = ["climate change", "inflation", "sleep quality"]
    reports = run_sequential_batch(topics)

    for topic, report in zip(topics, reports):
        r_tokens = report["research"].tokens
        w_tokens = report["write"].tokens
        ratio = r_tokens / w_tokens
        print(f"{topic}: research={r_tokens}t write={w_tokens}t")
        print(f"    ratio={ratio:.1f}x")
```

Expected output (exact numbers vary by run):

```
climate change: research=210t write=96t
    ratio=2.2x
inflation: research=198t write=88t
    ratio=2.3x
sleep quality: research=224t write=101t
    ratio=2.2x
```

Research consistently costs roughly 2x what write costs here — it needs a longer, more detailed prompt to gather enough material, while writing a short summary from already-gathered material is a shorter, cheaper call. This same `topics` list and the same `run_sequential_batch` shape is what `supervisor_compare` runs through the supervisor graph next, so the two sets of numbers line up topic-for-topic.

Difference between Approach 1 and Approaches 2/3: Approach 1's `main()` measures one topic and prints once — correct, but not reusable, and it can't tell you whether research is reliably more expensive or whether that one run was a fluke. Approach 2 restructures the same measurement into a typed, reusable shape (`StageResult` , `run_sequential()`) with no new behavior — the necessary refactor before this can be compared against anything else. Approach 3 uses that reusable shape across 3 topics to turn "research feels more expensive" into a specific, repeatable

number (a $\sim 2x$ ratio), which is what actually calibrates your intuition instead of leaving it as a vague impression.

Which one should you actually write? Approach 1 is genuinely fine for a one-off measurement you're about to throw away. Move to Approach 2's `StageResult` structure the moment you plan to compare this run against the supervisor version — you'll want the exact same shape from both, and a loose tuple or dict gets confusing fast once you're comparing two runs side by side. Approach 3's multi-topic ratio is worth running at least once per project, even if you don't keep the code around afterward — a specific number ("research costs 2x write") is something you can act on (cache research results, use a cheaper model for write), where "research feels more expensive" isn't.